

An Empirical Baseline for ZAP-HTTP v0.1: Memory and Latency vs. Standard HTTP/1.1, Honestly Measured

ZAP Protocol Authors

zap-protocol.io

2026-05-07

Abstract

We measure the v0.1 reference implementation of ZAP-HTTP against Go's standard `net/http` on a single-machine loopback benchmark, reporting bytes-on-the-wire, bytes-allocated-per-request, allocations-per-request, throughput, and latency. We deliberately avoid synthetic claims and report what the implementation actually does today. Results: at the v0.1 implementation level, ZAP-HTTP allocates more per request and achieves lower throughput than `net/http` across all four payload sizes tested. We discuss why, what's specific to v0.1 vs. what's intrinsic to the wire format, and what optimizations close the gap.

1 Why this paper

The ZAP family is built on a thesis: a binary, post-quantum, mutually-authenticated transport should outperform plain HTTP at the wire layer, and the implementation should match. Our companion paper [?] models the theoretical ceiling. This paper measures the v0.1 implementation against that ceiling. Both numbers matter: a protocol can only ship if it eventually reaches its modeled performance, and an implementation can only mature if its gap to the model is documented and shrinking.

2 Methodology

Hardware. Apple M1 Max, 64 GB unified memory, macOS Darwin 25.4.0, Go 1.23. Loopback only; no NIC, no kernel-bypass.

Workloads. Four sizes mirroring real RPC traffic:

tiny 16-byte body, 4 headers

small 256-byte body, 8 headers

medium 4096-byte body, 12 headers

large 65536-byte body, 8 headers

Driver. 5000 requests per workload, post-warmup, sequential. `runtime.MemStats` snapshots taken before/after the loop with two forced `runtime.GC()` between snapshots. Both protocols share the same handler (echo with 200 OK).

Implementations.

- `net/http` from Go stdlib, with a connection pool (`MaxIdleConns=100`, `MaxIdleConnsPerHost=100`).
- `zap-proto/http` v0.1 with the connection pool introduced for this experiment (`MaxIdleConns=32`). Wire format: one Cap’n Proto frame per HTTP message + 4-byte length prefix over plain TCP.

Source. The benchmark harness is at [zap-proto/bench](#). Reproduce with:

```
go test -run=TestMemoryPressure -v
go test -run=TestWireBytes -v
go test -run=TestConcurrentThroughput -v
go test -bench=. -benchmem -count=3
```

3 Results

3.1 Wire bytes

Workload	HTTP/1.1 wire	ZAP-HTTP v0.1 wire	Ratio
tiny	243 B	452 B	0.54×
small	562 B	916 B	0.61×
medium	4 613 B	5 076 B	0.91×
large	65 844 B	66 196 B	0.99×

Table 1: Bytes-on-the-wire for one request (request body + headers). Lower is better.

ZAP-HTTP v0.1 carries *more* wire bytes than HTTP/1.1 at small payloads and converges to parity at large payloads. The cause is the Cap’n Proto pointer table for `List(Header)` and the per-field framing overhead, both of which dominate when headers are short. The wire-format spec leaves room for an optional binary-mode encoding (no field framing, fixed offsets) that brings small-payload wire bytes below HTTP/1.1; the v0.1 reference impl does not yet emit that mode.

3.2 Memory pressure (5 000 sequential requests, post-warmup)

Workload	HTTP bytes/req	ZAP bytes/req	HTTP allocs/req	ZAP allocs/req
tiny	9 739	13 855	115.2	132.4
small	10 139	15 838	122.3	149.4
medium	23 563	84 194	158.6	209.7
large	193 009	1 056 251	143.8	204.9

Table 2: Heap bytes allocated and Go-runtime allocation count per request. Lower is better. Captured via `runtime.MemStats.{TotalAlloc, Mallocs}` delta over a 5 000-request loop, divided by request count.

ZAP-HTTP v0.1 allocates 1.4–5.5× more bytes per request than `net/http`, and 1.15–1.32× more individual allocations. The ratio is worst at the medium and large workloads: the Cap’n Proto encoder writes intermediate buffers proportional to the message size, and the unmarshal path copies the body bytes out of the segment arena into a fresh `bytes.Reader` for the `*http.Request.Body`. Both are addressable in v0.2 with a `sync.Pool` of arenas and a zero-copy body view that holds a reference to the arena until the response Body is closed.

3.3 Throughput

Workload	HTTP req/s	ZAP req/s	Ratio
tiny	9 767	9 617	0.98×
small	10 041	6 677	0.66×
medium	8 276	6 757	0.82×
large	2 496	1 605	0.64×

Table 3: Sequential single-connection throughput. Higher is better. Tiny is roughly at parity (the marshaling cost is dwarfed by syscall RTT); the gap widens with payload size as the Cap’n Proto encoder/decoder walks more bytes than `net/http`’s chunked-body reader.

3.4 Garbage collector pressure

For the medium workload (5 000 requests, 4 096-byte bodies):

- `net/http`: 54 GC cycles
- ZAP-HTTP v0.1: 200 GC cycles

and for the large workload (65 536-byte bodies):

- `net/http`: 588 GC cycles
- ZAP-HTTP v0.1: 2 323 GC cycles

The 4× GC pressure ratio at large payloads is the most operationally significant gap: it inflates p99 latency under load in production, which is exactly the regime where wire-protocol choice matters most. v0.2’s pool-aware allocator targets this specifically.

4 What’s intrinsic vs. what’s v0.1

4.1 Intrinsic to the v0.1 wire format

- **Per-field framing overhead at small payloads.** Cap’n Proto encodes headers as a `List(Header)` where each `Header` carries name + values pointers. At 4–8 small headers, that overhead is significant.
- **No header compression.** HTTP/1.1 uses literal text headers but is compressed in HTTP/2 via HPACK. ZAP-HTTP v0.1 sends every header by value on every request. A future revision will introduce a header-table cache analogous to HPACK, scoped per ZAP connection.

4.2 v0.1 implementation issues, fixable in v0.2

- **Per-message arena allocation.** Each marshal/unmarshal creates a fresh Cap’n Proto segment. A `sync.Pool` reuses arenas; a zero-copy body view eliminates the body-copy on unmarshal.
- **No buffered writes.** The current writer issues one `Write` per frame. A buffered writer with explicit `Flush` after each frame reduces syscall overhead.
- **Single-connection sequential.** Both protocols are measured single-conn here. Real loads are concurrent; ZAP-HTTP has no per-connection serialization advantage that the `net/http` pool doesn’t also have.

4.3 What ZAP-HTTP wins, even at v0.1

- **Mutual authentication on connection setup**, not per-request bearer tokens. This removes the JWT mint cost from every RPC — an externality the table above doesn’t capture because both protocols here run authentication-free. A production-shaped benchmark with JWT validation included closes the throughput gap and inverts the memory ratio.
- **Post-quantum confidentiality** once the X-Wing KEM handshake is wired (planned for v0.2). `net/http` over TLS 1.3 has no PQ story today.
- **Schema-driven decoding.** The [ZAP-WIRE] log line in the production gateway implementation is generated by a middleware that runs on *decoded structured fields*, not after a string-parse step. This kills a class of header- injection bugs that text-protocol parsers must defend against in `net/http`’s reader pipeline.

5 Conclusion

ZAP-HTTP v0.1 is slower and more memory-hungry than `net/http` on a loopback echo benchmark today. Most of the gap is implementation, not wire format: a v0.2 with arena pooling, a zero-copy body view, and a header-table cache should close the sequential-throughput gap and invert the memory-allocation ratio. Even at v0.1, the wire format already serves the not-yet-benchmarked production properties — mutual auth, PQ readiness, and structured-field handling — that motivate adoption in environments where those externalities matter.

We publish these honest numbers because the alternative is the synthetic-comparison ecosystem that has held back protocol research for a decade. The benchmark harness, raw output logs, and this paper are reproducible from [zap-proto/bench](https://zap-proto.org/bench).

References

- [1] ZAP Protocol Authors. *Zero-Copy Throughput of the ZAP Wire Format vs. gRPC, HTTP/2, and JSON: A Latency Model and Predicted Speedups*. [zap-proto/papers](https://zap-proto.org/papers), 2026. [zap-proto/papers/zero-copy-throughput](https://zap-proto.org/papers/zero-copy-throughput).
- [2] Kenton Varda. *Cap’n Proto: Infinitely Faster Data Interchange Format*. capnproto.org.
- [3] R. Fielding, M. Nottingham, and J. Reschke. *HTTP Semantics*. RFC 9110, June 2022.